LA-MTL: Latency-Aware Automated Multi-Task Learning

Shambhavi Balamuthu Sampath[1,2†] Sami Sawani[1†] Moritz Thoma[1,2] Lukas Frickenstein[2]
Pierpaolo Mori[2,3] Nael Fasfous[2] Manoj Rohit Vemparala[2] Alexander Frickenstein[2]
Ulf Schlichtmann[1] Claudio Passerone[3] Walter Stechele[1]

† Equal Contribution [1]Technical University of Munich [2]BMW Group [3]Politecnico Di Torino

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132930

***Abstract*—Multi-Task Learning (MTL) aims to unify a variety of tasks into a single network for improved training and inference efficiency. This is particularly attractive for real-time applications that require simultaneous execution of multiple workloads in resource-constrained embedded environments. However, most MTL approaches focus on enhancing parameters efficiency and overall tasks metrics, often lacking explicit inference latency awareness in the optimization loop. The design space exploration should not compromise on the parameters efficiency or task accuracy objectives in order to meet latency requirements. To address this, we propose LA-MTL, an automated layer-level MTL policy search that incorporates a novel analytical latency factor (ALF). By accounting for local and global latencies during the MTL policy search, we derive solutions that balance task metrics, parameters efficiency and latency constraints. LA-MTL search on ResNet34 yields solutions with up to 50% lower latency on the Jetson AGX Orin while maintaining competitive metrics in semantic segmentation and depth estimation tasks with a +/-2 p.p., on the CityScapes dataset. Additionally, we achieve a superior parameters efficiency, surpassing the state-of-the-art MTL parameters reduction by over 20 p.p. Experiments on benchmark datasets (CityScapes, NYUv2) demonstrate the effectiveness of our approach across various backbones including ResNet34, MobileNetV2, and MobileOne in its expanded form. Code is available at https://github.com/shamvbs/LA-MTL.** [1]

***Index Terms*—multitask learning, convolutional neural networks (CNNs), edge devices, gradients conflict, tasks interference, proxy latency.**

I. Introduction

As the number of tasks and model size increases, memory footprint and computational complexity also grow, making deployment on resource-constrained devices increasingly difficult. This challenge has driven the adoption of multitask learning (MTL), where a single model is trained to perform multiple tasks simultaneously. By sharing layers across tasks, classic MTL offers significant parameters reduction without sacrificing accuracy. However, these benefits can be insufficient for latency-sensitive applications such as autonomous driving, where rapid processing is crucial. A vehicle must detect and locate objects, estimate distances, and perform other functions in real-time. The effect of latency reduction on parameters efficiency and task accuracy is often hard to predict, since they are not directly proportional to each other. In our quest to incorporate latency awareness into MTL, we ask the following questions. *(1) Is there a positive synergistic relationship between latency optimization and classic MTL goals, and can we leverage this? (2) If tasks accuracy or parameters efficiency is sacrificed for faster inference, can these losses be recovered?* The answer to these questions may very well change for different backbones and/or tasks. Therefore, there is a need for a latency-aware optimization scheme that is able to reflect latency penalty for various configurations. At the same time, dealing with these concerns can be cumbersome and time-consuming if not conveniently automated as a part of one cohesive latency-aware MTL framework. To address these challenges, we introduce LA-MTL, with the following contributions:

- ***Analytical Latency Factor:*** A novel latency proxy metric, capturing both local and global context of the multi-task model using a unitless mapping cost at the layer level.
- A ***latency loss*** to adhere to the latency constraints, that respects a user-defined analytical latency requirement based on the target latency.
- ***Gradients/capacity modification mechanism*** for modifying gradients and capacity of shared layers across different tasks to minimize task interference and improve overall task performance.

II. Related Works

a) MTL for Dense Predictions: Various strategies have been used to build MTL solutions. Manual adjustments, which rely on domain expertise, often prove suboptimal due to the difficulty in identifying the optimal configuration within a typically large search space [1] [2]. Another approach is policy learning through Neural Architecture Search (NAS), which involves determining specific neural network depth and width as policy decisions. While NAS introduces a degree of automation, its implementation can vary; some approaches restrict the backbone capacity [3], while others are more adaptive [4], facilitating trade-offs between performance and resource usage. Following [4], our framework, LA-MTL allows the MTL model capacity to dynamically change based on the tasks needs.

b) Hardware-Aware Neural Architecture Search (HW-aware NAS): Hardware-awareness in vision MTL has often been overlooked. Nevertheless, the concept of HW-aware NAS has been extensively explored outside of vision MTL to optimize for device-specific constraints. Proxy-based latency-aware NAS methods typically suffer from poor generaliza-

[1]Correspondence to: shambhavi.balamuthu-sampath@tum.de

tion across diverse hardware architectures [5]–[7]. Some approaches directly incorporate real hardware metrics such as latency into the NAS process [8]–[12]. Additionally, lookup tables (LUTs) are pre-built in certain methods to capture hardware measurements, which can reduce the need for costly, real-time hardware-in-the-loop setups [13]. Even the vision MTL contributions that incorporate latency awareness [14] [15], often do so while being tightly coupled to a specific architectures, HW, and design choices- which necessitates significant manual fine-tuning efforts to extend to other backbones and design space encodings. LA-MTL, on the other hand, requires minimal changes to work with, and supports any convolutional architecture to be optimized for edge deployment.

c) Tasks Interference and Gradients Conflict: An orthogonal MTL strategy involves viewing MTL as a multi-objective optimization problem aimed at minimizing task interference [16]. Effective parameters sharing can induce positive inductive biases between tasks, but excessive sharing can cause gradients conflict. In gradient descent algorithms, the optimization of one task gradients can inadvertently affect the other tasks' gradients moving them into a worse solution point in the design space. Many works modify gradients magnitude and direction to alleviate the problem [17] [18] [16].

Often, MTL training pipeline involves several stages. Interference does not occur during every optimization step. Therefore, it is essential not only to manage task conflicts but also to identify when they occur. Interference is also influenced by the model architecture used in the search space. Our LA-MTL approach facilitates the handling of gradients at different training stages, thereby improving the management of MTL conflicts for a given setup.

Existing SOTA methods that are most relevant to our approach are presented in table I. AutoMTL [4] supports any convolutional backbone with minimal effort but lacks gradient conflict handling and latency-awareness. muNet [19] eliminates conflicts by updating parameters for one task at a time but is not latency-aware. MGD [16] handles interference by deriving Pareto optimal solutions but requires manual integration and is not latency-aware. MTNAS [20] includes latency-awareness but lacks conflict handling and requires adaptation for different backbones and devices. In contrast, our method supports any convolutional backbone with minimal modifications, automatically handles operator-level gradient conflicts, and incorporates latency-awareness. This combination of features ensures efficient and effective multi-task learning across different setups, setting our method apart from current SOTA approaches.

Related Work	Gradients Conflict Handling	CNN-Backbone-Agnostic	Latency Awareness
Auto-MTL [4]	✗	✓	✗
muNet [19]	✓	✓	✗
MGD [16]	✓	✗	✗
MTNAS [20]	✗	✗	✓
LA-MTL (Ours)	✓	✓	✓

TABLE I: Overview of Related Works vs. LA-MTL.

III. LA-MTL Framework

We design and implement a Latency-Aware MTL solution to ensure that the inference latency constraints are met while maintaining competitive task metrics performance. With every layer in a given backbone containing operations like convolution or batch normalization, we follow the operator-level search space encoding from AutoMTL [4]. For the set of N tasks denoted as $\mathbf{T} = \{t_1, t_2, ..., t_N\}$, each task t_i can either share the original backbone operator, has its own copy, or skip that layer's operator (unity or down-sampling). As shown in Fig. 1, such an encoding provides diverse and flexible modulations of a given backbone.

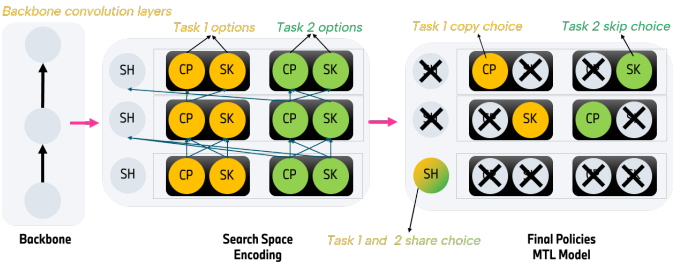

Fig. 1: AutoMTL [4] exponential search space with operator-level encoding. The differentiable search algorithm learns a Gumbel-Softmax distribution and samples an MTL model with fixed branching decisions.

We search for good branching decisions, by learning a branching policy distribution to arrive at a parameters efficient, low latency, and competitive accuracy solution. The overall pipeline is illustrated in Fig. 2. The training pipeline is split into three phases: In the first warm-up phase, no policy training is involved. Rather, we feed forward the average of the original operator, copied operator, and skip connection. The second stage jointly optimizes the sharing policy and the model parameters. Then, a multi-task model is generated by sampling from the learned policy distribution. A final third training stage is performed where the model parameters are trained from scratch while the branching policy is fixed.

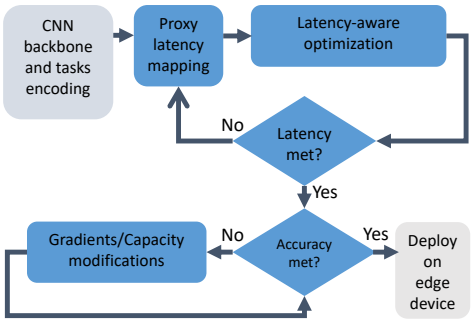

Fig. 2: The main components of our work: We encode our proxy ALF at the operator level. Then, we optimize latency-aware loss with optional gradients/ capacity adjustments.

A. Analytical Latency Factor: A Proxy Latency

Proxy metrics like FLOPs can be used to approximate actual latency, but they do not always correlate well. A reduction in FLOPs can also lead to an increase in latency instead of a decrease, depending on the model's architecture and hardware constraints. Yet, when working well, proxies remain attractive due to their minimum overhead during MTL search space exploration/optimization. To address this, we propose the *Analytical Latency Factor* (ALF), a flexible, unitless proxy metric for latency. For all backbone architectures and task configurations, ALF mapping is adjusted accordingly. The ALF is initially applied uniformly across all layers as follows:

- Original Convolution: To have a lower latency than the single-task approach using the given backbone, we model the original convolution operator as having maximum mapping cost with an ALF factor of 1.
- Task Copy: Since it is the same operation as the original layer but with independent parameters, we also assign it an ALF of 1.
- Skip Connection: A skip connection that applies a unity operation analytically costs 0. If the skip connection down-samples, the mapping becomes of 0.5.

The initial mapping above is obtained from empirical latency profiling of individual layers' forward pass. We recognize that this coarse-grained approach may not always accurately reflect the true computational/latency cost, especially in cases where the choice of operations introduces complex inter-dependencies between layers. To address this, we allow granular control over the latency factor mapping, particularly for cases where skipping a layer can reduce the computational load for that layer but may increase the size of the tensor passed to subsequent layers, potentially leading to orders of magnitude more FLOPs and increased latency. We define down-sampled convolutions as 1x1 convolution kernels of the original backbone operator. For example, we retain the original depthwise or dilated convolutions but down-sample them, unlike [4] that always forces to standard 1x1 convolution while down-sampling. The ALF is implemented in two main contexts:

- *Local ALF*: this factor sums up the latency costs associated with each operator/layer in a given task's path.
- *Global ALF*: this factor captures the interactions between multiple tasks. The global ALF considers the combined inference paths of all tasks, encouraging configurations that enable parallel task execution.

The global ALF does not interfere with the local optimality of each task's analytical latency, but rather, it encourages simultaneous activation of task paths for a given hardware that is able to leverage such inference paths. Fig. 3 illustrates local/ global ALF concept with an example calculation.

B. Hardware-Aware Loss

For $\mathcal{N}$ tasks and $\mathcal{L}$ layers, The chosen policy choice for task i will be denoted by $\mathcal{P}_l^{t_i}$. This is a simple discrete variable that is 0 if the backbone operator is chosen, 1 if the task-specific copy is chosen, or 2 for skip connection. The Gumbel-Softmax approximation is denoted by the below policy variable:

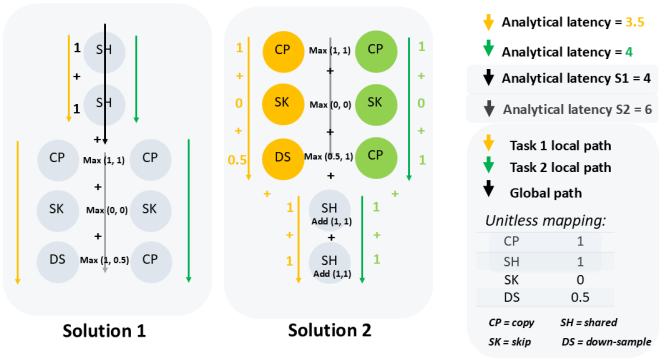

Fig. 3: Local and Global Analytical Latency Factor Mapping. Both solutions have the same local paths analytical latency. The global analytical latency prefers solution 1 since it can benefit from hardware parallelization without necessarily affecting parameters efficiency or individual tasks' policy decisions.

$$\mathcal{P}'(k) = \frac{\exp((G_k + \log(\pi_k))/\tau)}{\sum_{k\in\{0,1,2\}} \exp((G_k + \log(\pi_k))/\tau)} \tag{1}$$

where $k \in \{0, 1, 2\}$ represents the three operator options. The standard Gumbel distribution [21] $G_k \sim Gumbel(0, 1)$ is used to draw i.i.d (independent and identically distributed) samples where $\pi = [\pi_0, \pi_1, \pi_2]$ is the policy distribution ($\pi_0 + \pi_1 + \pi_2 = 1$), and τ is the temperature parameter. We propose a dynamic objective function that leverages both local and global ALF to guide policy-decisions training. The objective is to minimize:

$$\mathcal{L}_{total} = \sum \lambda_i \mathcal{L}_i + \lambda_{reg} \cdot \left(\alpha \cdot L_{Lat.} + (1 - \alpha) \cdot L_{reg}\right) \tag{2}$$

where λ_i are task weights, $\mathcal{L}_i$ are the individual task losses, and λ_{reg} is a scalar regularizer. To dynamically adjust the focus between latency optimization and parameters efficiency, we introduce an adaptive coefficient, α. If a user-defined local ALF requirement is met, we set α to zero, allowing $\mathcal{L}_{reg}$ to focus on parameters efficiency (through layers-sharing promotion). Otherwise, we set α to 1 which activates the latency-aware term, $L_{Lat.}$, penalizing policies exceeding analytical latency requirements. The latency-focused term is:

$$\mathcal{L}_{Lat.} = \sum_{i\leq N} \sum_{l\leq L} \left\{ \mathcal{GL} * \left(\exp^{\mathcal{P}_l^{t_i}(1)+\mathcal{P}_l^{t_i}(0)-\mathcal{P}_l^{t_i}(2)}\right) \right\} \tag{3}$$

$\mathcal{GL}$ is the global ALF. As gradient-descent step is applied to each task separately, the global factor can capture the latest changes from previous tasks' updates and inform the next task for a better global analytical latency. The term discourages copying and sharing the original operator. Instead,

it opts for aggressive down-sampling and unity. Meanwhile, the parameters efficient term [4] is:

$$\mathcal{L}_{reg} = \sum_{i \leq N} \sum_{l \leq L} \frac{L-l}{L} \Bigg\{ \ln\big(1 + \exp^{\mathcal{P}_l^{t_i}(1) - \mathcal{P}_l^{t_i}(0)}\big) + \ln\big(1 + \exp^{\mathcal{P}_l^{t_i}(2) - \mathcal{P}_l^{t_i}(0)}\big) \Bigg\} \tag{4}$$

The exponential nature of $\mathcal{L}_{Lat.}$ leads to aggressive updates during the initial training phase, driving the model towards low-analytical-latency configurations, but potentially sacrificing accuracy. Once the analytical latency requirement is met, the more linear updates from $\mathcal{L}_{reg}$ allow the model to gradually explore solutions and progressively increase capacity. As training progresses, $\mathcal{L}_{reg}$ may eventually push the model beyond the analytical latency limit, triggering a reactivation of $\mathcal{L}_{Lat.}$. This brings the model back into the low-latency design space region, but with a different configuration—one that has a higher starting capacity and better accuracy than before. This iterative process allows for deeper exploration of solutions near the aggressive low-analytical-latency boundary.

C. Capacity Increase and Conflict Handling

The Conflict Definition: Equation (2) can suffer from potential tasks interference. Let us assume $K \geq 2$ distinct tasks, each associated with a loss function $\mathcal{L}_i(\theta)$, where θ represents a shared set of parameters. The objective is to find an optimal θ that minimizes losses across all tasks. Let us also assume that $\sum_i \lambda_i = 1$. Taking the first term of (2), we have

$$\theta^* = \arg\min_{\theta \in \mathbb{R}^m} \left\{ L_0(\theta) \triangleq \frac{1}{K} \sum_{i=1}^{K} L_i(\theta) \right\}. \tag{5}$$

Let $g_i = \nabla L_i(\theta)$ represent the gradient for task i. The averaged gradient across all tasks becomes $g_0 = \nabla L_0(\theta) = \frac{1}{K} \sum_{i=1}^{K} g_i$. Using a learning rate $\alpha \in \mathbb{R}^+$, the steepest descent update is $\theta \leftarrow \theta - \alpha g_0$. However, g_0 can conflict with individual gradients, meaning that for some task i, $\langle g_i, g_0 \rangle < 0$. When this conflict is significant, g_0 can degrade performance on task i. To handle interference, we combine a mitigation and removal strategy. The latter option is only possible after the second phase of training where the policies are finalized. Before starting the third phase, we calculate conflict scores for each convolutional layer shared between two or more tasks. Then, we allow up to 'n' layers to be transformed from shared to task-specific. By using task-specific layers, the gradients conflict is completely removed. The conflict scores are calculated by doing forward pass and back-propagation for $\mathcal{I}$ iterations. Mathematically, we utilize a layer-wise S-conflict score [22]. Let the shared model parameters θ_{sh} consist of n layers, i.e., $\theta_{\text{sh}} = \{\theta_{\text{sh}}^{(k)}\}_{k=1}^{n}$, where $\theta_{\text{sh}}^{(k)}$ represents the k^{th} shared layer. The gradient of task i with respect to the k^{th} shared layer, denoted by $\mathbf{g}_i^{(k)}$, is the vector of partial derivatives of $\mathcal{L}_i$ with respect to the parameters of $\theta_{\text{sh}}^{(k)}$. We Let $\phi_{ij}^{(k)}$ represent the angle between $\mathbf{g}_i^{(k)}$ and $\mathbf{g}_j^{(k)}$. The definition for layer conflict and S-conflict score as found in [22] is:

Definition 1 (Layer-wise Conflicting Gradients): The gradients $\mathbf{g}_i^{(k)}$ and $\mathbf{g}_j^{(k)}$ $(i \neq j)$ are considered conflicting if $\cos \phi_{ij}^{(k)} < 0$.

Definition 2 (S-Conflict Score): For any $-1 < S \leq 0$, the S-conflict score for the k^{th} shared layer is the number of task pairs (i, j) $(i \neq j)$ such that $\cos \phi_{ij}^{(k)} < S$, denoted as $s^{(k)}$. The top 'n' S-conflict score layers will be transformed to task-specific. The remaining shared layers will be handled through gradients modifications as in [23]. We find an update vector within a local region centered around the average gradient g_0 of the shared layers added to our overall latency-Aware loss, while minimizing conflict among tasks.

While we combine elements from both [23] and [22], our approach differs in: *1)* our loss is not an average gradient with equal weights but rather, a dynamic one that changes based on the analytical latency requirements. *2)* the gradients conflict handling is done on a layer level and is compatible with not only pure hard parameter-sharing backbones but also for backbones with various branching decisions where sharing occurs and disappear in different parts of the architecture. *3)* we enforce task-specific copies for BatchNorm layers. *4)* conflict handling in shared layers is allowed at any phase of the three training stages. *5)* in the third training stage, we train model parameters from scratch.

IV. Evaluation

For our experiments, we follow AutoMTL [4] for all evaluation metrics, task losses, hyperparameters, and data augmentation. On CityScapes [24] and NYUv2 [25] datasets, we optimize Deeplab models with ResNet-34 [26] and MobileNetV2 [27] backbones, and Atrous Spatial Pyramid Pooling (ASPP) task-specific heads. We additionally evaluate on the expanded form of the structurally reparametrizable MobileOne [28] backbone.

Backbone	Approach	Sem.Segm. ↑			Depth Est. ↑			All tasks ↑			Params ↓	
		ONNX	TRT	QNN	ONNX	TRT	QNN	ONNX	TRT	QNN	Rel.	Rel2.
ResNet-34 [26]	Baseline										-49.01	-48.19
	+LA-MTL-C	**15**	**23**	**18**	**15**	**11**	-49	**20**	**22**	**19**	**-68.88**	-68.14
	+LA-MTL-A	**40**	**45**	**27**	**28**	**34**	**20**	**33**	**39**	**33**	**-76.03**	-75.22
MobileNetV2 [27]	Baseline										-45.68	+4.43
	+LA-MTL-C	-11	**2**	1	-15	-17	**9**	-1	**2**	**3**	**-68.01**	+10.83
	+LA-MTL-A	-11	0	-19	0	**11**	1	**1**	**3**	**4**	**-87.91**	+14.22

TABLE II: **Latency Optimization on CityScapes**. Latency speedup and parameters achieved with LA-MTL (A: Aggressive, C: Conservative) compared to the AutoMTL baseline [4].

.

Backbone	Approach	Sem.Segm. ↑			Depth Est. ↑			Normal ↑			All tasks ↑			Params ↓	
		QNNX	TRT	QNN	QNNX	TRT	QNN	QNNX	TRT	QNN	QNNX	TRT	QNN	Rel.	Rel2.
ResNet-34 [26]	Baseline													-49.25	-48.47
	+LA-MTL-C	**19**	**21**	**20**	**26**	**27**	**32**	-2	**6**	0	**19**	**20**	**18**	**-62.02**	-61.06
	+LA-MTL-A	**35**	**39**	**32**	**44**	**62**	**61**	**18**	**40**	**34**	**44**	**48**	**44**	**-84.77**	-83.27
MobileNetV2 [27]	Baseline													-40.46	-10.52
	+LA-MTL-C	-3	-3	**1**	**1**	**20**	**6**	**15**	-18	**1**	**2**	**1**	**4**	**-75.75**	+6.44
	+LA-MTL-A	-23	**1**	0	**5**	-14	-2	-16	-28	**3**	**7**	-4	**6**	**-82.76**	+12.05

TABLE III: **Latency Optimization on NYUv2**. Latency speedup and parameters achieved with LA-MTL (A: Aggressive, C: Conservative) compared to the AutoMTL baseline [4].

For the parameters reduction metric, we calculate it relative to a *single task approach* in two ways. 'Rel.' follows the SOTA [4] by convention, and considers share-choice and copy-choice only and 'Rel2.' considers share-choice, copy-choice,

and skip-choice. We utilize two variants of our latency-aware loss. The conservative variant sets a latency target of 0.5 times the analytical latency of the original backbone for each task. In contrast, the aggressive variant imposes a stricter target, aiming for 0.2 times the original backbone's analytical latency. These latency constraints are applied uniformly across all tasks and layers, along with a consistent analytical mapping for all layers and tasks. However, we permit individual layer mappings to deviate from this uniform mapping if the direct neighborhood of a layer has significantly higher FLOPs, differing by orders of magnitude.

A. Latency Optimization

We measure the latency (in milliseconds) of a single experiment by averaging 20 consecutive inferences of the MTL backbones (excluding task heads). Deployments are executed on two distinct platforms: Nvidia Jetson AGX Orin [29] using ONNX/TensorRT runtime, and Qualcomm Snapdragon Gen 2 mobile development kit [30] utilizing Qualcomm AI Direct SDK QNN runtime [31]. Tables II and III present the reduction metrics for latency and parameters on ResNet-34 and MobileNetV2 backbones, on CityScapes and NYUv2 datasets, respectively. The observations from Table II on CityScapes and Table III on NYUv2 provide comprehensive insights into the performance of LA-MTL variants using ResNet-34 and MobileNetV2 backbones. For CityScapes, the conservative approach (LA-MTL-C) with ResNet-34 achieves consistent latency reductions across most tasks and runtimes, with parameter reductions up to 68.88%, though there is a notable 49% latency increase for the QNN runtime in depth estimation. The aggressive approach (LA-MTL-A) demonstrates even greater latency reductions, especially for semantic segmentation and depth estimation, achieving up to 45% faster latency and parameter reductions of 76.03%. For MobileNetV2, the conservative approach shows mixed latency results but achieves a parameter reduction of 68.01%, while the aggressive approach indicates higher latencies for individual tasks but faster overall latency when all tasks are activated simultaneously, achieving the highest parameter reduction of 87.91%. For the NYUv2 dataset, the conservative approach (LA-MTL-C) with ResNet-34 shows latency reductions across semantic segmentation, depth estimation, and normal tasks, with parameter reductions of 62.02%. The aggressive approach (LA-MTL-A) achieves even more significant latency reductions, with up to 62% reduction for depth estimation and 40% for normal tasks, and parameter reductions of 84.77%. For MobileNetV2, the conservative approach again shows mixed results, with some latency increases, but achieves a parameter reduction of 75.75%. The aggressive approach shows higher latencies for individual tasks but achieves up to 82.76% parameter reduction. These results showcase the effectiveness of LA-MTL approaches in optimizing latency and memory footprint across different datasets, backbones and runtimes. The conservative approach generally provides balanced improvements, while the aggressive approach can achieve more substantial reductions but may involve trade-offs in individual task latencies. These outcomes can be attributed to several factors, including the blackbox compiler optimizations, which can vary in their effectiveness depending on the obtained MTL architecture. Additionally, utilization of hardware resources is crucial, as different optimization strategies may lead to varying degrees of hardware utilization efficiency. This highlights the importance of choosing the appropriate optimization strategy based on the target platform, runtime and performance goals.

B. Task Metrics and SOTA Comparison

Optimizing for latency often directly impacts task metrics, potentially degrading them. Therefore, our focus extends to further boosting task metrics performance achieved with LA-MTL. For ResNet-34, the non-latency-aware AutoMTL [4] loss clearly benefited from interference handling and increased capacity (Table VI) across all task metrics. The results from the table indicate that the gradient-modified AutoMTL model achieves superior performance in semantic segmentation compared to other state-of-the-art (SOTA) models. Specifically, it achieves a mean Intersection over Union (mIoU) of 44.63% and pixel accuracy of 75.07%. This performance surpasses other well-known models such as**Cross-Stitch** [2], **Sluice** [32], **NDDR-CNN** [33], **MTAN** [1], **DEN**, [34], and **AdaShare** [3], which report lower values for mIoU and pixel accuracy. The gradient-modified AutoMTL model not only excels in accuracy but also manages to maintain a relatively low parameter count, making it an efficient choice for multi-task learning applications. Additionally, the LA-MTL-C model demonstrates remarkable parameter efficiency, achieving a parameter reduction of -68.88% (Rel.) and -68.14% (Rel2.), while still maintaining competitive performance in both semantic segmentation and depth estimation tasks. This level of parameter reduction is the highest among the models compared, indicating that LA-MTL-C is highly efficient in terms of resource usage. For MobileNetV2, the gradient-modified LA-MTL-C model particularly stands out with the best depth estimation accuracy, achieving a 1.25 accuracy of 46.51. This performance is achieved while also maintaining significant parameter reductions, showcasing the model's ability to balance between performance and efficiency. In contrast, other models like AdaShare and the original AutoMTL, while showing strong performance, do not achieve the same level of parameter efficiency. Overall, these comparisons, also proved in VII highlight the effectiveness of the proposed approach in achieving a balance between high task performance and parameter efficiency. The gradient-modified AutoMTL and LA-MTL-C models, in particular, outperform other SOTA methods in various metrics and tasks, making them highly suitable for practical applications where resource constraints are a concern.

C. Ablation Study

Table V shows that our global factor is able to promote further parameters efficiency, faster local paths latency, and competitive metrics. As for tasks metrics, they were competitive to baseline (drop $< 1\%$) and surpassed it in $\delta 1.25^2$

CityScapes	Params Reduction		Sem. Segm.					Depth Estimation					
	Rel. ↑	Rel2 ↑	mIoU ↑	P.Acc ↑	Lat1^a ↑	Lat2^a ↑	Lat3^a ↑	1.25 ↑	1.25^2 ↑	1.25^3 ↑	Lat1^a ↑	Lat2^a ↑	Lat3^a ↑
MobileOne													
AutoMTL [4]	35.07	**21.57**	**25.20**	**63.62**	0.0	0.0	0.0	**46.18**	**74.51**	**87.15**	0.0	0.0	0.0
LA-MTL-C	49.65	29.22	24.60	63.45	-7	-7	-1	45.31	73.92	86.93	10	-7	-2
LA-MTL-A	**66.05**	**38.32**	24.36	63.38	9	16	4	33.87	57.33	74.40	16	-49	21

TABLE IV: MobileOne expanded form with no capacity/gradients modifications on CityScapes dataset. Values are an average of 4 random seeds. All metrics in %; Latencies in ms; ↑ = higher is better. P.Acc = Pixel Acc.; Lat1, Lat2, Lat3 = ONNX, TRT, QNN latency reduction.

Method	Params Reduction ↑		Semantic Seg.			Depth Estimation					
						Error ↓		δ, within ↑			
	Rel. (%)	Rel2. (%)	mIoU ↑	Pixel Acc. ↑	Lat1. ↓	Abs.	Rel.	1.25	1.25^2	1.25^3	Lat1. ↓
Conserv. w/ GAL	33.30	33.21	33.47	71.63	25.46	0.0232	0.399	64.42	83.11	91.40	24.43
Conserv. w/o GAL	23.86	23.84	34.06	**71.81**	26.49	**0.0223**	0.4343	64.24	82.29	90.47	26.49
Aggres. w/ GAL	**52.47**	**52.05**	33.03	71.53	**21.60**	0.0219	0.4116	**64.47**	82.73	90.95	**21.60**
Aggres. w/o GAL	31.67	31.56	**34.16**	71.67	25.04	0.0237	**0.3864**	64.30	**83.64**	**91.62**	25.06
AutoMTL [4]	26.54	26.23	33.71	71.76	28.07	0.0229	0.3957	64.61	83.48	91.51	28.11
Conserv. w/ GAL	**37.86**	**36.66**	**32.64**	**71.55**	22.04	**0.0222**	0.4171	**63.52**	**82.32**	**90.61**	**20.74**
Conserv. w/o GAL	26.24	25.36	31.58	71.42	22.13	0.0225	0.4385	62.75	81.47	90.05	22.12
Aggres. w/ GAL	44.96	43.69	31.74	71.42	**18.71**	0.0227	0.4287	62.53	81.61	90.21	21.21
Aggres. w/o GAL	34.56	33.68	32.51	71.58	22.01	0.0228	0.4519	62.97	81.07	89.50	22.35
AutoMTL [4]	36.11	35.71	33.69	71.65	25.64	0.0229	**0.4113**	64.40	83.08	91.01	25.76
Conserv. w/ GAL	**36.13**	**35.42**	**32.94**	**71.84**	29.56	0.0225	0.4361	63.21	82.12	90.46	29.63
Conserv. w/o GAL	31.28	30.57	32.59	71.76	29.81	0.0231	0.3981	63.77	**83.12**	**91.30**	**27.10**
Aggres. w/ GAL	39.99	38.97	31.38	71.62	**27.06**	0.0245	0.4145	62.20	82.26	90.84	27.34
Aggres. w/o GAL	36.39	35.68	31.55	71.52	29.27	**0.0222**	0.4159	63.82	82.49	90.67	29.93
AutoMTL [4]	44.50	44.11	33.35	71.71	30.43	**0.0216**	**0.4476**	**64.20**	81.90	89.92	31.26

TABLE V: Global ALF Effect (CityScapes, Resnet34) and ONNX latencies in ms.

and $\delta 1.25^3$. Our ALF method is able to deliver- again-latency improvement in average for the expanded MobileOne backbone (Table IV). While mostly losing less than 1 p.p. on tasks metrics, we achieved up to 21% faster latency and simultaneously removed 38.32% parameters compared to just 21.57% baseline.

CityScapes	# Params ↓		Sem. Segm.		Depth Estimation		
	Rel. (%)	Rel2. (%)	mIoU ↑	Pixel Acc. ↑	1.25 ↑	1.25^2 ↑	1.25^3 ↑
ResNet-34							
Single-Task	-	-	36.5	73.8	57.5	76.9	87.0
Multi-Task	-50.0	-	42.7	68.1	58.8	80.5	89.9
Cross-Stitch [2]	+0.0	-	40.3	74.3	70.0	86.3	93.1
Sluice [32]	+0.0	-	39.8	74.2	68.9	85.8	92.8
NDDR-CNN [33]	+3.5	-	41.5	74.2	69.9	86.3	93.0
MTAN [1]	+20.5	-	40.8	74.3	71.0	86.3	92.8
DEN [34]	-44.0	-	38.0	74.2	68.2	84.5	91.6
AdaShare [3]	-50.0	-	40.6	74.7	**71.4**	**86.8**	93.1
AutoMTL [4]	-32.3	-	42.8	74.8	70.0	86.6	**93.4**
+ Grad. Mod.	-41.38	-40.28	**44.63**	**75.07**	70.11	86.16	92.98
LA-MTL-C	-68.88	-68.14	39.31	74.27	66.42	84.23	91.98
+ Grad. Mod.	-58.17	-57.41	42.07	74.59	69.19	84.78	92.04
LA-MTL-A	**-76.03**	**-75.22**	39.21	74.22	63.67	83.31	91.65
MobileNetV2							
Single-Task	-	-	25.9	63.5	32.1	71.1	86.5
Multi-Task	-50.0	-	26.3	63.4	33.6	66.5	82.9
AdaShare [3]	-50.0	-	**26.7**	61.2	42.1	71.6	84.3
AutoMTL [4]	-33.5	-	25.8	**63.7**	44.4	**74.8**	**87.3**
LA-MTL-C	**-66.55**	+14.51	22.81	63.2	39.57	65.98	80.32
+ Grad. Mod.	-50.95	+7.25	24.01	63.03	**46.51**	72.02	85.3
LA-MTL-A	**-90.50**	+14.16	25.99	62.55	39.95	62.91	77.06

TABLE VI: SOTA Comparison on the CityScapes dataset with LA-MTL.

V. Conclusion

This paper proposed a latency-aware MTL optimization method that attempts maintaining good tasks performance, parameters efficiency, and lower latency for edge inference. For any given convolutional architecture, our modular framework

CityScapes	# Params ↓		Sem. Segm.		Depth Estimation			Normal		
	Rel.(%)	Rel2.(%)	mIoU ↑	Pixel Acc. ↑	δ, within ↑			θ, within ↑		
					1.25	1.25^2	1.25^3	θ11°	θ22°	θ30°
ResNet-34										
Single-Task	-	-	26.5	58.2	57.8	85.8	96.0	29.4	**72.3**	**87.3**
Multi-Task	- 67.68	-	22.2	54.4	60.9	87.7	**96.7**	32.2	70.5	84.8
Cross-Stitch	+0.0	-	25.4	57.6	61.4	88.4	95.5	41.4	67.7	80.4
Sluice	+0.0	-	23.8	56.9	61.9	88.1	96.3	38.9	69.0	81.4
NDDR-CNN	+1.81	-	21.6	53.9	55.7	83.7	94.8	37.4	70.9	83.1
MTAN	+0.55	-	26.0	57.2	**62.7**	87.7	95.9	**43.7**	70.5	81.9
DEN	-63.80	-	23.9	54.9	22.8	62.4	88.2	36.0	70.6	83.4
AdaShare	-67.68	-	24.4	57.8	61.3	**88.5**	96.5	42.3	68.9	80.5
AutoMTL	-50.64	-49.94	26.33	57.97	61.29	87.77	96.38	37.76	70.68	83.57
LA-MTL-C	-71.85	-70.88	**26.92**	**58.58**	50.96	80.99	93.74	38.52	**72.58**	85.11
LA-MTL-A	**-87.91**	**-86.41**	23.79	55.53	49.31	79.68	93.02	32.79	72.17	86.40
MobileNetV2										
AutoMTL [4]	-40.46	**-10.52**	**18.69**	**47.52**	**53.85**	**82.76**	**94.22**	**27.10**	72.04	86.65
LA-MTL-C	-75.75	+6.44	17.70	46.67	51.08	81.12	93.49	26.44	72.10	86.77
LA-MTL-A	**-82.76**	+12.05	17.17	46.40	47.53	77.07	91.17	20.50	**72.27**	**87.03**

TABLE VII: Performance comparison across various multi-task learning methods.

is designed to work independently with little manual effort. The modest capacity increase along with gradients conflict mitigation boosted SOTA AutoMTL to have improved task metrics outperforming other related works. The latency aware optimization along with the analytical latency factor delivered a superior parameters efficiency surpassing SOTA. Tested on Cityscapes and NYUv2 datasets and on Resnet, MobileNet, and MobileOne architectures, LA-MTL is indeed able to navigate through the design space while finding solutions with improved latency and competitive task metrics.

References

[1] S. Liu, E. Johns, and A. J. Davison, "End-to-end multi-task learning with attention," in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 1871–1880.

[2] I. Misra, A. Shrivastava, A. Gupta, and M. Hebert, "Cross-stitch networks for multi-task learning," 2016.

[3] X. Sun, R. Panda, R. Feris, and K. Saenko, "Adashare: Learning what to share for efficient deep multi-task learning," 2020.

[4] L. Zhang, X. Liu, and H. Guan, "Automtl: A programming framework for automating efficient multi-task learning," 2022.

[5] A. M. Garavagno, E. Ragusa, A. Frisoli, and P. Gastaldo, "Running hardware-aware neural architecture search on embedded devices under 512mb of ram," *IEEE International Conference on Consumer Electronics*, 2024.

[6] L. L. Zhang, Y. Yang, Y. Jiang, W. Zhu, and Y. Liu, "Fast hardware-aware neural architecture search," 2020.

[7] H. Bouzidi, H. Ouarnoughi, S. Niar, and A. A. E. Cadi, "Performance prediction for convolutional neural networks in edge devices," 2020.

[8] M. Tan, B. Chen, R. Pang, V. Vasudevan, M. Sandler, A. Howard, and Q. V. Le, "Mnasnet: Platform-aware neural architecture search for mobile," 2019.

[9] N. Sinha, A. E. R. Shabayek, A. Kacem, P. Rostami, C. Shneider, and D. Aouada, "Hardware aware evolutionary neural architecture search using representation similarity metric," *arXiv.org*, 2023.

[10] Z. Li, M. Paolieri, and L. Golubchik, "Inference latency prediction at the edge," 2022.

[11] H. Cai, L. Zhu, and S. Han, "Proxylessnas: Direct neural architecture search on target task and hardware," 2019.

[12] S. Zeng, H. Sun, Y. Xing, X. Ning, Y. Shan, X. Chen, Y. Wang, and H. Yang, "Black box search space profiling for accelerator-aware neural architecture search," in *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2020, pp. 518–523.
[13] H. Lee, S. Lee, S. Chong, and S. J. Hwang, "Help: Hardware-adaptive efficient latency prediction for nas via meta-learning," 2021.
[14] M. Tan and Q. V. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," 2020. [Online]. Available: https://arxiv.org/abs/1905.11946
[15] H. Liang, Z. Fan, R. Sarkar, Z. Jiang, T. Chen, K. Zou, Y. Cheng, C. Hao, and Z. Wang, "M^{3}vit: Mixture-of-experts vision transformer for efficient multi-task learning with model-accelerator co-design," 2022. [Online]. Available: https://arxiv.org/abs/2210.14793
[16] X. Zhou, Y. Gao, C. Li, and Z. Huang, "A multiple gradient descent design for multi-task learning on edge computing: Multi-objective machine learning approach," *IEEE Transactions on Network Science and Engineering*, vol. 9, no. 1, pp. 121–133, 2022.
[17] A. Javaloy and I. Valera, "Rotograd: Gradient homogenization in multitask learning," 2022. [Online]. Available: https://arxiv.org/abs/2103.02631
[18] T. Yu, S. Kumar, A. Gupta, S. Levine, K. Hausman, and C. Finn, "Gradient surgery for multi-task learning," 2020. [Online]. Available: https://arxiv.org/abs/2001.06782
[19] A. Gesmundo and J. Dean, "munet: Evolving pretrained deep neural networks into scalable auto-tuning multitask systems," 2022.
[20] T. Vu, Y. Zhou, C. Wen, Y. Li, and J. Frahm, "Toward edge-efficient dense predictions with synergistic multi-task neural architecture search," *IEEE Workshop/Winter Conference on Applications of Computer Vision*, 2023.
[21] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with gumbel-softmax," 2017. [Online]. Available: https://arxiv.org/abs/1611.01144
[22] G. Shi, Q. Li, W. Zhang, J. Chen, and X.-M. Wu, "Recon: Reducing conflicting gradients from the root for multi-task learning," 2023. [Online]. Available: https://arxiv.org/abs/2302.11289
[23] B. Liu, X. Liu, X. Jin, P. Stone, and Q. Liu, "Conflict-averse gradient descent for multi-task learning," 2024. [Online]. Available: https://arxiv.org/abs/2110.14048
[24] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, and B. Schiele, "The cityscapes dataset for semantic urban scene understanding," *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3213–3223, 2016. [Online]. Available: https://api.semanticscholar.org/CorpusID:502946
[25] N. Silberman, D. Hoiem, P. Kohli, and R. Fergus, "Indoor segmentation and support inference from rgbd images," in *Proceedings of the 12th European Conference on Computer Vision - Volume Part V*, ser. ECCV'12. Berlin, Heidelberg: Springer-Verlag, 2012, p. 746–760. [Online]. Available: https://doi.org/10.1007/978-3-642-33715-4_54
[26] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille, "Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs," 2017. [Online]. Available: https://arxiv.org/abs/1606.00915
[27] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 4510–4520.
[28] P. K. A. Vasu, J. Gabriel, J. Zhu, O. Tuzel, and A. Ranjan, "Mobileone: An improved one millisecond mobile backbone," 2023. [Online]. Available: https://arxiv.org/abs/2206.04040
[29] NVIDIA, "Nvidia jetson orin," https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/?srsltid=AfmBOoo4VBZk9BMaYyKmT7nvGsMOhavegQB0W-db77zi7l3qj5WZVrXz, 2022, accessed: 2023-04-12.
[30] Qualcomm, "Snapdragon 8 gen 2 mobile platform," https://www.qualcomm.com/products/mobile/snapdragon/smartphones/snapdragon-8-series-mobile-platforms/snapdragon-8-gen-2-mobile-platform, 2022, accessed: 2023-04-12.
[31] Q. Technologies, "Snapdragon® 8 gen 2 mobile hardware development kit," https://developer.qualcomm.com/hardware/snapdragon-8-gen-2-hdk, 2024.
[32] S. Ruder, J. Bingel, I. Augenstein, and A. Søgaard, "Latent multi-task architecture learning," in *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence and Thirty-First Innovative Applications of Artificial Intelligence Conference and Ninth AAAI Symposium on Educational Advances in Artificial Intelligence*, ser. AAAI'19/IAAI'19/EAAI'19. AAAI Press, 2019. [Online]. Available: https://doi.org/10.1609/aaai.v33i01.33014822
[33] Y. Gao, J. Ma, M. Zhao, W. Liu, and A. L. Yuille, "Nddr-cnn: Layerwise feature fusing in multi-task cnns by neural discriminative dimensionality reduction," in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 3200–3209.
[34] C. Ahn, E. Kim, and S. Oh, "Deep elastic networks with model selection for multi-task learning," 2019. [Online]. Available: https://arxiv.org/abs/1909.04860